

UniVoz — Sistema Colaborativo com Apoio à Democracia Eletrônica

Universidade Federal dos Vales do Jequitinhonha e Mucuri (UFVJM) Departamento de Computação
— Curso: Sistemas de Informação Disciplina: Sistemas Colaborativos

Equipe:

Membro	Papel
Gabriel Castro Guimarães	Backend
Humberto Freire Pereira	Frontend
Pávila Miranda Cardoso	Frontend
Victor Ryan Vieira	DevOps
Murilo Santiago Escobedo	QA

Entregáveis:

- Texto completo: este documento (também em PDF)
- Slides: **UniVoz.pdf** (apresentação do projeto)
- Repositório dos fontes: <https://github.com/4snt/SOCIAL-APP> (fork de desenvolvimento)
- Repositório base: <https://github.com/GabGC0608/SOCIAL-APP>
- Demonstração em produção: <https://univoz.flipfile.com>

1. Identificação do Projeto

Tema: Sistema colaborativo de votação de propostas acadêmicas.

Problema: A comunidade acadêmica (alunos, professores e técnicos) muitas vezes não possui um canal estruturado, transparente e acessível para sugerir melhorias ou participar das decisões da faculdade. As opiniões acabam não sendo ouvidas de forma organizada, dificultando a participação democrática.

Justificativa: A criação de um sistema colaborativo de votação permite aumentar o engajamento da comunidade acadêmica, promovendo transparência e participação ativa nas decisões. Além disso, facilita a coleta de ideias, a priorização de demandas e a comunicação entre alunos e gestão.

Objetivo: Desenvolver uma plataforma onde usuários possam cadastrar propostas, votar (curtir), comentar e acompanhar os resultados, promovendo um ambiente colaborativo e democrático dentro da instituição.

Nível de Participação Cidadã (Gomes):

- **Nível 2 — Consulta de opinião:** a comunidade cadastra propostas, vota e comenta.
- **Nível 3 — Prestação de contas:** a universidade (perfil institucional/admin) altera o estado de cada demanda (*pendente* → *em andamento* → *concluída*), tornando o processo transparente e auditável.

2. Modelo 3C de Colaboração

2.1 Ferramentas usadas pela equipe no desenvolvimento

Dimensão	Ferramenta
Comunicação	Discord e WhatsApp
Coordenação	Trello
Cooperação	GitHub (repositório base + fork, commits, merges entre upstream e fork)

2.2 Modelo 3C aplicado ao sistema

- **Comunicação:** usuários escrevem em caixas de comentários e se comunicam sobre os problemas relatados (`CommentController`, componente `CommentList`).
- **Coordenação:** através da votação (likes) e dos comentários, os usuários e a gestão (PROAD/universidade) coordenam a prioridade das demandas; o feed pode ser ordenado por engajamento (`GET /api/posts?sortBy=likes`).
- **Cooperação:** usuários cooperam ao apoiar propostas de outros estudantes, sugerir melhorias nas propostas já feitas através dos comentários, propor novas demandas e votar nas que julgam mais importantes.

3. Requisitos para Democracia Eletrônica

Requisito	Como o sistema atende
Colaboração	Pelo Modelo 3C, os usuários colaboram entre si e com a universidade: relatam problemas, votam, comentam e propõem melhorias.
Transparência	A universidade altera o estado da solicitação (<i>pendente</i> , <i>em andamento</i> , <i>concluída</i>) via painel administrativo; o status é exibido publicamente em cada proposta (<code>StatusBadge</code>). Propostas com mais engajamento podem ser consultadas ordenando o feed por votos.
Memória	O sistema armazena todo o histórico de propostas, votos e comentários — inclusive propostas já concluídas — para que haja memória do que já esteve em pauta e do que foi executado. O log de atividades (<code>ActivityLog</code>) registra as ações relevantes.

4. Atividade 02 — Percepção e Contexto

4.1 Funcionalidades de percepção propostas × implementação

Funcionalidade proposta	Situação	Implementação
Notificações automáticas sobre alterações importantes	Sim	Entidade <code>Notification</code> , <code>NotificationController</code> (<code>GET /api/notifications</code> , <code>GET /api/notifications/unread-count</code> , <code>PUT /api/notifications/{id}/read</code>) e componente <code>NotificationMenu</code> no header.
Histórico de ações realizadas pelos usuários	Sim	Entidade <code>ActivityLog</code> + <code>ActivityLogService</code> ; consulta via <code>GET /api/admin/activity</code> .
Registro de atividades e interações do grupo	Sim	Log de atividades registra criação de posts, comentários, likes e ações de moderação.

Funcionalidade proposta	Situação	Implementação
Indicação de status das tarefas (pendente, em andamento, concluída)	Sim	Campo <code>status</code> em <code>Post</code> , badge visual (<code>StatusBadge</code>) e moderação via PUT <code>/api/admin/posts/{postId}/status</code> .
Alertas de atualização em tempo real	Parcial	Percepção via notificações e contador de não lidas (polling); WebSocket/SSE registrado como melhoria futura no <code>PLANO_IMPLEMENTACAO.md</code> .
Exibição de usuários online e offline	Planejado	Previsto no plano de implementação (Fase 1/2).
Identificação do foco de atenção dos participantes	Planejado	Previsto no plano de implementação.

4.2 Funcionalidades de contexto propostas × implementação

Funcionalidade proposta	Situação	Implementação
Recuperação de contexto histórico para auxiliar decisões	Sim	Feed permanente com todas as propostas (inclusive concluídas); histórico de comentários e votos por proposta; log de atividades para o admin.
Organização automática de informações conforme prioridade	Sim	Ordenação do feed por engajamento/data (<code>sortBy=likes createdAt</code>), destacando as demandas mais votadas.
Sugestão de ações com base no comportamento dos participantes	Parcial	O painel administrativo agrega atividade recente e demandas pendentes, orientando a ação da gestão.
Uso de localização geográfica quando necessário	Planejado	Previsto no plano de implementação (Fase 3).
Adaptação das notificações conforme horário/atividade	Planejado	Previsto no plano de implementação (Fase 3).
Monitoramento do ambiente virtual	Parcial	Dashboard administrativo com visão geral de usuários, demandas e atividade.

5. Atividade 03 — Usabilidade, Comunicabilidade e Sociabilidade

5.1 Requisitos de Usabilidade

Requisito	Implementação
Interface simples e intuitiva	UI em Next.js + Tailwind com paleta própria (<code>brand-*</code>) e componentes padronizados (<code>btn-primary</code> , <code>card</code> , <code>input-field</code>).

Requisito	Implementação
Facilidade de navegação	Header fixo com acesso direto a feed, perfil, notificações e painel admin (quando aplicável).
Tempo de resposta rápido	API REST enxuta; build otimizado do Next.js (multi-stage Docker, output standalone).
Compatibilidade mobile e desktop	Layout responsivo com Tailwind (grid/flex, breakpoints).
Facilidade de aprendizado	Fluxos curtos (cadastro → propor → votar/comentar); página de ajuda (/help).
Mensagens de erro claras	<code>GlobalExceptionHandler</code> no backend padroniza erros; feedback inline nos formulários.
Organização visual adequada	Cards de proposta com autor, status, contadores de votos e comentários.
Acesso rápido às funcionalidades principais	Criar proposta e votar diretamente no feed, sem trocar de página.
Redução de passos nas tarefas	Like em um clique (idempotente); comentário na própria página do post.
Ajuda/orientações ao usuário	Página /help com orientações de uso.

5.2 Requisitos de Comunicabilidade

Requisito	Implementação
Linguagem clara e objetiva	Toda a interface em PT-BR com termos do domínio (proposta, demanda, votação).
Feedback imediato após ações	Contadores de like/comentário atualizam na hora; estados de carregamento nos botões.
Notificações compreensíveis	<code>NotificationMenu</code> com mensagens descritivas e contador de não lidas.
Ícones e elementos visuais claros	Ícones de coração (voto), balão (comentário) e badges de status coloridos.
Indicação clara do status das atividades	<code>StatusBadge</code> (<i>pendente / em andamento / concluída</i>) em cada proposta.
Identificação dos responsáveis por ações	Autor exibido em cada post/comentário com <code>UserBadge</code> de papel (estudante, universidade, admin).
Facilidade de compartilhamento	Página pública por proposta (/post/[id]) e por usuário (/u/[username]).
Histórico de interações acessível	Comentários e votos permanentes por proposta; log de atividades no painel admin.

5.3 Requisitos de Sociabilidade

Requisito	Implementação
Interação entre usuários	Comentários, respostas e votos entre membros da comunidade.
Suporte à colaboração em grupo	Propostas abertas a toda a comunidade; discussão coletiva antes/depois da votação.
Compartilhamento de informações e arquivos	Upload de imagens nas propostas (<code>multipart/form-data</code> , armazenamento BYTEA).

Requisito	Implementação
Controle de permissões e papéis	Herança de usuários (<code>STUDENT</code> , <code>UNIVERSITY</code> , <code>ADMIN</code>) com <code>SINGLE_TABLE</code> + discriminador; gestão de papéis via <code>PUT /api/admin/users/{userId}/role</code> .
Registro das contribuições	Cada proposta, voto e comentário fica vinculado ao autor.
Notificações sobre atividades do grupo	Sistema de notificações sobre interações nas propostas do usuário.
Histórico colaborativo das ações	<code>ActivityLog</code> + memória permanente do feed.
Comunicação síncrona e assíncrona	Assíncrona via comentários/notificações no app; síncrona planejada (<code>WebSocket/SSE</code>) e suprida pela equipe via Discord/WhatsApp.

6. Arquitetura e Tecnologia

6.1 Stack

Camada	Tecnologia	Responsabilidade
Frontend	Next.js 14 (App Router) + Tailwind CSS	UI, rotas, chamadas REST com <code>credentials: 'include'</code>
Backend	Spring Boot 3 + Spring Security + JPA/Hibernate	API REST, autenticação por sessão, regras de negócio
Banco	PostgreSQL 16	Persistência relacional (inclusive imagens em BYTEA)
Orquestração	Docker Compose (3 contêineres)	Build e execução local/produção
Deploy	Coolify + Traefik	Publicação em https://univoz.flipafle.com

```

Usuário --HTTP--> Next.js (frontend :3000)
                    | /api/* (proxy, cookie JSESSIONID)
                    v
                Spring Boot (backend :8080)
                    | JDBC
                    v
                PostgreSQL 16 (db :5432)

```

6.2 Backend em camadas

Padrão Controller → Service → Repository → Entity:

- **Controllers:** `AuthController`, `PostController`, `CommentController`, `LikeController`, `UserController`, `NotificationController`, `AdminController`, `AdminDashboardController`.
- **Services:** `AuthService`, `PostService`, `CommentService`, `LikeService`, `UserService`, `NotificationService`, `ActivityLogService`.
- **Entities:** `User` (+ subclasses `StudentsUser`, `UniversityUser`, `AdminUser`), `Post`, `Comment`, `Like`, `Notification`, `ActivityLog`.
- **Segurança:** autenticação por sessão (`HttpSession` + cookie `JSESSIONID`), senhas com `BCrypt`, CORS configurável por variável de ambiente (`CORS_ALLOWED_ORIGINS`), tratamento global de erros com `GlobalExceptionHandler`.

6.3 Modelo de dados

```
users (SINGLE_TABLE, dtype: USER|STUDENT|UNIVERSITY|ADMIN)
  +--< posts (image_data BYTEA, description, status, created_at)
  |   +--< comments (content, created_at)
  |   +--< likes (UNIQUE(user_id, post_id))
  +--< comments
  +--< likes
  +--< notifications (message, read, created_at)
  +--< activity_log (ação, autor, data)
```

- Herança de usuários com @Inheritance(SINGLE_TABLE) e coluna discriminadora dtype — todos os perfis compartilham a tabela users.
- likes possui restrição UNIQUE(user_id, post_id): **cada usuário vota uma única vez por proposta** (requisito de votação).
- Imagens das propostas persistidas no banco como BYTEA e servidas como data URL base64 ou por GET /api/posts/{postId}/image.

6.4 API REST (resumo)

Grupo	Endpoints principais
Autenticação	POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me
Propostas	GET /api/posts (público, filtros sortBy/direction/userId), GET /api/posts/{id}, POST /api/posts (multipart), GET /api/posts/{id}/image
Votação	POST /api/posts/{id}/like, DELETE /api/posts/{id}/like
Comentários	GET/POST /api/posts/{id}/comments, DELETE /api/comments/{id}
Usuários	GET /api/users/{id}, GET /api/users/by-username/{username}
Notificações	GET /api/notifications, GET /api/notifications/unread-count, PUT /api/notifications/{id}/read
Administração	PUT /api/admin/posts/{postId}/status, GET /api/admin/users, PUT /api/admin/users/{userId}/role, GET /api/admin/activity

Referência completa com exemplos de request/response em docs/API.md.

7. Histórico de Desenvolvimento (repositório base → fork)

O desenvolvimento partiu do repositório base criado por Gabriel (<https://github.com/GabGC0608/SOCIAL-APP>) e evoluiu no fork (<https://github.com/4snt/SOCIAL-APP>), com merges periódicos entre os dois — a própria colaboração da equipe usou o GitHub como ferramenta de **Cooperação** do Modelo 3C.

Fase 1 — MVP do feed social (repo base)

92efe69 Initial commit → 8450586 → 3bb0723 funcionando com cadastro automatico → 7122554 → 1b7a15f/a8cce7e ordenar feed

Feed básico com posts, cadastro de usuários e ordenação.

Fase 2 — Redesign e interação social

f604db2 feat: redesign social com Tailwind + feed público + comentários

Interface refeita com Tailwind, feed público para visitantes, sistema de comentários.

Fase 3 — Perfis de usuário e autenticação por sessão

33c39f6 classes filhas de user e uso de cookies sem ID na URL

Herança de usuários (estudante, universidade, admin) e autenticação por sessão com cookies — base para o controle de papéis exigido pela sociabilidade.

Fase 4 — Documentação e upload de imagens

1a6931e docs: add architecture and backlog documentation → 1054b5e upload de fotos usando BYTEA → 5616fbb status da tarefa

Documentação de arquitetura/backlog, upload de imagens por multipart/form-data com persistência em BYTEA e primeiro campo de status das demandas.

Fase 5 — DevOps e produção

3918fe6 → 25d9ebe → 808eed9 fix: remove hardcoded hosts/ports → 1ee2ed8 fix: remove credenciais hardcodadas → 5ae339c perf: multi-stage build → 853a550 → 4b0f69e → 2c62449 → f7e4890 → fa60bc4 fix: CORS + /api/auth/me

Configuração 100% por variáveis de ambiente, Dockerfiles otimizados e deploy no Coolify — sistema publicado em <https://univoz.flipafle.com>.

Fase 6 — Identidade UniVoz e moderação

46a1ab4 feat: tema univoz no seed inicial → 3a6953f feat: admin panel for demand moderation

Seed com o tema de demandas acadêmicas da UFVJM e painel administrativo para moderação de demandas.

Fase 7 — Percepção, contexto e papéis (Atividades 2 e 3)

926994e feat: implement activity logging and admin dashboard features → 1406595 fix: clear do banco → a6eba0a feat: enhance moderation and collaboration feedback → 4a20b3e feat: add admin role management and contextual features

Log de atividades, dashboard administrativo, notificações, gestão de papéis de usuário e melhorias de feedback de colaboração — implementando os requisitos de percepção, contexto, usabilidade, comunicabilidade e sociabilidade das Atividades 2 e 3.

8. Como Executar

Local (Docker Compose)

```
# 1. copie o template de variáveis de ambiente
cp .env.example .env
```

2. ajuste os valores em .env se necessário (defaults já funcionam)

3. suba os containers

docker compose up --build

Serviço	URL padrão
Frontend	http://localhost:6003
Backend	http://localhost:6001/api/posts
Postgres	localhost:6002 (db socialdb, user admin)

As portas do host são configuráveis via DB_HOST_PORT, BACKEND_HOST_PORT e FRONTEND_HOST_PORT no .env.

Usuários de seed têm senha 123 (ex.: gabriel@email.com, maria@email.com).

Deploy no Coolify

No Coolify, configure as seguintes variáveis de ambiente pelo painel (as portas do host não são necessárias — o Traefik gerencia o roteamento):

Variável	Descrição	Exemplo prod
POSTGRES_DB	Nome do banco	socialdb
POSTGRES_USER	Usuário do banco	admin
POSTGRES_PASSWORD	Senha do banco	<senha segura>
SPRING_DATASOURCE_URL	URL JDBC interna	jdbc:postgresql://db:5432/socialdb
CORS_ALLOWED_ORIGINS	Domínios permitidos pelo CORS	https://univoz.flipafire.com
NEXT_PUBLIC_API_URL	URL interna do backend (server-side)	http://backend:8080

Estrutura do repositório

```
.
+-- backend/          # Spring Boot - controllers, services, entities, repositories
|   +-- src/main/java/com/example/social/
+-- frontend/         # Next.js - app/, components/, hooks/, lib/
+-- docs/             # Documentação técnica
|   +-- ARCHITECTURE.md # diagramas (Mermaid), camadas, fluxos
|   +-- API.md          # referência REST
|   +-- BACKLOG_SOCIAL_APP.md # épicos e user stories
+-- PLANO_IMPLEMENTACAO.md # plano das funcionalidades das Atividades 2 e 3
+-- UniVoz.pdf         # slides da apresentação
+-- docker-compose.yaml
```

9. Conclusão

O UniVoz materializa os conceitos da disciplina em um sistema funcional e publicado: o **Modelo 3C** aparece tanto no processo da equipe (Discord/WhatsApp, Trello, GitHub) quanto no produto (comentários, votação com priorização e cooperação em propostas); os requisitos de **democracia eletrônica** são atendidos com transparência (status das demandas moderado pela universidade), memória (histórico permanente de propostas, votos, comentários e log de atividades) e participação nos níveis 2 e 3 de Gomes (consulta de opinião e prestação de contas).

As funcionalidades de **percepção e contexto** (Atividade 2) foram implementadas por meio de notificações, log de atividades, status das demandas e ordenação por engajamento; os requisitos de **usabilidade, comunicabilidade e sociabilidade** (Atividade 3) orientaram a interface responsiva, o feedback imediato, os badges de status e papel, a página de ajuda e o controle de permissões por papéis. Os itens restantes (presença online, tempo real via WebSocket, geolocalização) estão priorizados no `PLANO_IMPLEMENTACAO.md` como evolução natural do sistema.

Anexos

Documento	Arquivo
Slides da apresentação	UniVoz.pdf
Documento das Atividades 1–3 (definição do projeto)	Atv 3 - Sistema Colaborativo com Apoio à DE.pdf
Análise 3C e proposta de funcionalidades	Sistemas Colaborativos (2).pdf
Arquitetura técnica (diagramas Mermaid)	docs/ARCHITECTURE.md
Referência da API REST	docs/API.md
Backlog (épicas e user stories)	docs/BACKLOG_SOCIAL_APP.md
Plano de implementação	PLANO_IMPLEMENTACAO.md